

1. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Program :

```
#include <stdio.h>

int main() {
    int n,i,j,k=0,found;

    int a[100],b[100];

    printf("Enter number of elements: ");

    scanf("%d",&n);

    printf("Enter elements: ");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    for(i=0;i<n;i++){
        found=0;

        for(j=0;j<k;j++){
            if(a[i]==b[j]){
                found=1;

                break;
            }
        }

        if(found==0){
            b[k]=a[i];

            k++;
        }
    }

    printf("Unique elements: ");

    for(i=0;i<k;i++)
        printf("%d ",b[i]);

    return 0;
}
```

Input & Output :

```
Enter number of elements: 6
Enter elements: 5 5 2 4 4 1
Unique elements: 5 2 4 1

...Program finished with exit code 0
Press ENTER to exit console.
```

2. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

Program :

```
#include <stdio.h>

int main(){
    int n,i,j,temp;

    int a[100];

    printf("Enter number of elements: ");

    scanf("%d",&n);

    printf("Enter elements: ");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    for(i=0;i<n-1;i++){
        for(j=0;j<n-i-1;j++){
            if(a[j]>a[j+1]){
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
        }
    }

    printf("Sorted array: ");

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
```

```
}
```

Input and Output :

```
Enter number of elements: 5
Enter elements: 8 5 2 4 9
Sorted array: 2 4 5 8 9

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Program :

```
#include <stdio.h>

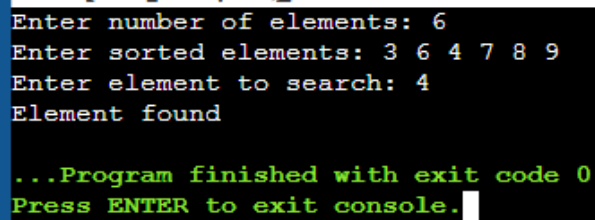
int main(){
    int n,i,x,low,high,mid,found=0;
    int a[100];
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter sorted elements: ");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("Enter element to search: ");
    scanf("%d",&x);
    low=0;
    high=n-1;
    while(low<=high){
        mid=(low+high)/2;
        if(a[mid]==x){
            found=1;
            break;
        }
        else if(a[mid]<x)
            low=mid+1;
        else
            high=mid-1;
    }
}
```

```

    }
    if(found==1)
        printf("Element found");
    else
        printf("Element not found");
    return 0;
}

```

Input & Output :



```

Enter number of elements: 6
Enter sorted elements: 3 6 4 7 8 9
Enter element to search: 4
Element found

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Program :

```

#include <stdio.h>

int main(){
    int n,i,j,k,temp;

    int a[100];

    printf("Enter number of elements: ");

    scanf("%d",&n);

    printf("Enter elements: ");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    for(i=n/2-1;i>=0;i--){
        j=i;
        while(2*j+1<n){
            k=2*j+1;

            if(k+1<n && a[k]<a[k+1])

                k++;

```

```

        if(a[j]>=a[k])
            break;
        temp=a[j];
        a[j]=a[k];
        a[k]=temp;
        j=k;
    }
}
for(i=n-1;i>0;i--){
    temp=a[0];
    a[0]=a[i];
    a[i]=temp;
    j=0;
    while(2*j+1<i){
        k=2*j+1;
        if(k+1<i && a[k]<a[k+1])
            k++;
        if(a[j]>=a[k])
            break;
        temp=a[j];
        a[j]=a[k];
        a[k]=temp;
        j=k;
    }
}
printf("Sorted array: ");
for(i=0;i<n;i++)
    printf("%d ",a[i]);
return 0;
}

```

Input & Output :

```
Enter number of elements: 7
Enter elements: 2 0 4 9 8 6 0
Sorted array: 0 0 2 4 6 8 9

...Program finished with exit code 0
Press ENTER to exit console.
```